Models of Generics and Metaprogramming: Go, Rust, Swift, D and More

In some domains of programming it's common to want to write a data structure or algorithm that can work with elements of many different types, such as a generic list or a sorting algorithm that only needs a comparison function. Different programming languages have come up with all sorts of solutions to this problem: From just pointing people to existing general features that can be useful for the purpose (e.g C, Go) to generics systems so powerful they become Turing-complete (e.g. Rust, C++). In this post I'm going to take you on a tour of the generics systems in many different languages and how they are implemented. I'll start from how languages without a special generics system like C solve the problem and then I'll show how gradually adding extensions in different directions leads to the systems found in other languages.

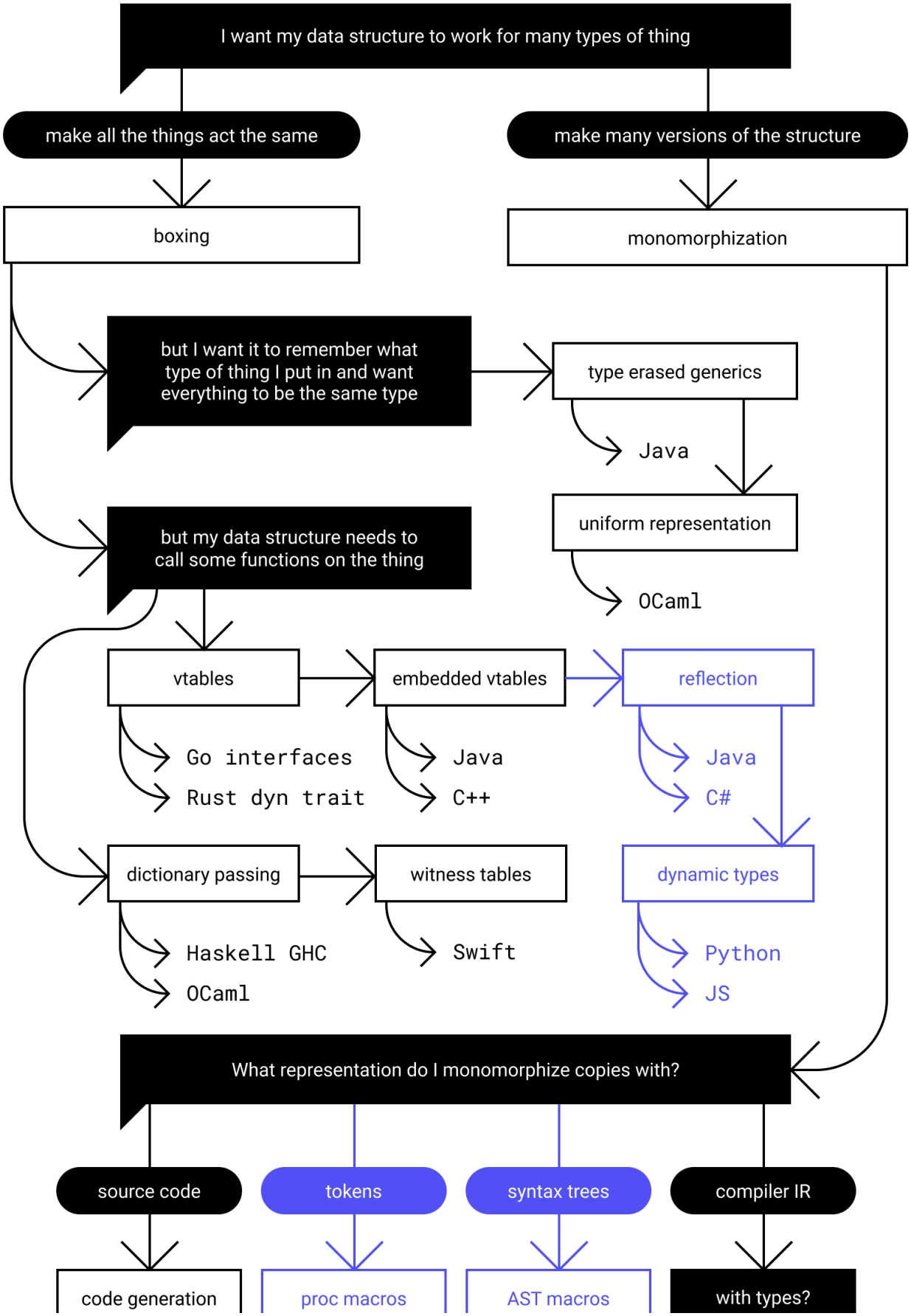
One reason I think generics are an interesting case is that they're a simple case of the general problem of metaprogramming: writing programs that can generate classes of other programs. As evidence I'll describe how three different fully general metaprogramming methods can be seen as extensions from different directions in the space of generics systems: dynamic languages like Python, procedural macro systems like Template Haskell, and staged compilation like Zig and Terra.

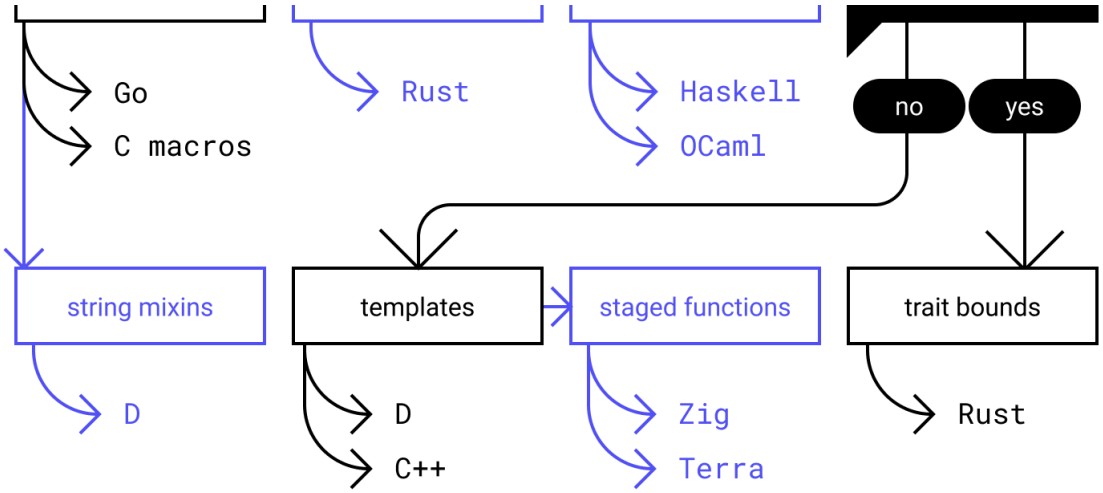
Overview

I made a flow chart of all the systems I discuss to give you an overview of what this post will contain and how everything fits together:

How Languages Implement Generics and Extensions to Metaprogramming

http://thume.ca/





The basic ideas

Let's say we're programming in a language without a generics system and we want to make a generic stack data structure which works for any data type. The problem is that each function and type definition we write only works for data that's the same size, is copied the same way, and generally acts the same way.

Two ideas for how to get around this are to find a way to make all data types act the same way in our data structure, or to make multiple copies of our data structure with slight tweaks to deal with each data type the correct way. These two ideas form the basis of the two major classes of solutions to generics: "boxing" and "monomorphization".

Boxing is where we put everything in uniform "boxes" so that they all act the same way. This is usually done by allocating things on the heap and just putting pointers in the data structure. We can make pointers to all different types act the same way so that the same code can deal with all data types! However this can come at the cost of extra memory allocation, dynamic lookups and cache misses. In C this corresponds to making your data structure store `void*` pointers and just casting your data to and from `void*` (allocating on the heap if the data isn't already on the heap).

Monomorphization is where we copy the code multiple times for the different types of data we want to store. This way each instance of the code can directly use the size and methods of the data it is working with, without any dynamic lookups. This produces the fastest possible code, but comes at the cost of bloat in code size and compile times as the same code with minor tweaks is compiled many times. In C this corresponds to defining your entire data structure in a macro and calling it for each type you want to use it with.

Overall the tradeoff is basically that boxing leads to better compile times but can hurt runtime performance, whereas monomorphization will generate the fastest code but takes extra time to compile and optimize all the different generated instances. They also differ in how they can be extended: Extensions to boxing allow more dynamic behavior at runtime, while monomorphization is more flexible with how different instances of generic code can differ. It's also worth noting that in some larger programs the performance advantage of monomorphization might be canceled out by the additional instruction cache misses from all the extra generated code.

Each of these schools of generics has many directions it can be extended in to add additional power or safety, and different languages have taken them in very interesting directions. Some languages like Rust and C# even provide both options!

Boxing

Let's start with an example of the basic boxing approach in Go:

```
type Stack struct {
  values []interface{}
}

func (this *Stack) Push(value interface{}) {
  this.values = append(this.values, value)
}

func (this *Stack) Pop() interface{} {
  x := this.values[len(this.values)-1]
  this.values = this.values[:len(this.values)-1]
  return x
}
```

Example languages that use basic boxing: C (`void*`), Go (`interface{}`), pre-generics Java (`Object`), pre-generics Objective-C (`id`)

Type-erased boxed generics

Here's some problems with the basic boxing approach:

- Depending on the language we often need to cast values to and from the correct type every time we read or write to the data structure.
- Nothing stops us from putting elements of different types into the structure, which could allow bugs that manifest as runtime crashes.

A solution to both of these problems is to add generics functionality to the type system, while still using the basic boxing method exactly as before at runtime. This approach is often called type erasure, because the types in the generics system are "erased" and all become the same type (like `Object`) under the hood.

Java and Objective-C both started out with basic boxing, and later added language features for generics with type erasure, even using the exact same collection types as before for compatibility, but with optional generic type parameters. See the following example from the Wikipedia article on Java Generics:

```
List v = new ArrayList();
v.add("test"); // A String that cannot be cast to an Integer
Integer i = (Integer)v.get(0); // Run time error
```

```
List<String> v = new ArrayList<String>();
v.add("test");
Integer i = v.get(0); // (type error) compilation-time error
```

Inferred boxed generics with a uniform representation

OCaml takes this idea even further with a uniform representation where there are no primitive types that require an additional boxing allocation (like `int` needing to be turned into an `Integer` to go in an `ArrayList` in Java), because everything is either already boxed or represented by a pointer-sized integer, so everything is one machine word. However when the garbage collector looks at data stored in generic structures it needs to tell pointers from integers, so integers are tagged using a 1 bit in a place where valid aligned pointers never have a 1 bit, leaving only 31 or 63 bits of range.

OCaml also has a type inference system so you can write a function and the compiler will infer the most generic type possible if you don't annotate it, which can lead to functions that look like a dynamically typed language:

```
let first (head :: tail) = head
(* inferred type: 'a list -> 'a *)
```

The inferred type is read as "a function from a list of elements of type `'a` to something of type `'a`". Which encodes the relation that the return type is the same as the list type but can be any type.

+ Interfaces

A different limitation in the basic boxing approach is that the boxed types are *completely* opaque. This is fine for data structures like a stack, but things like a generic sorting function need some extra functionality, like a type-specific comparison function. There's a number of different ways of both implementing this at runtime and exposing this in the language, which are technically different axes and you can implement the same language using multiple of these approaches. However, it seems like different language features mostly lend themselves towards being implemented a certain way, and then language extensions take advantage of the strengths of the chosen implementation. This means there's mostly two families of languages based around the different runtime approaches: vtables and dictionary passing.

Interface vtables

If we want to expose type-specific functions while sticking with the boxing strategy of a uniform way of working with everything, we can just make sure that there's a uniform way to find the type-specific function we want from an object. This approach is called using "vtables" (shortened from "virtual method tables" but nobody uses the full name) and how it is implemented is that at offset zero in every object in the generic structure is a pointer to some tables of function pointers with a consistent layout. These tables allow the generic code to look up a pointer to the type-specific functions in the same way for every type by indexing certain pointers at fixed offsets.

This is how `interface` types are implemented in Go and `dyn` `trait` objects are implemented in Rust. When you cast a type to an interface type for something it implements, it creates a wrapper that

contains a pointer to the original object and a pointer to a vtable of the type-specific functions for that interface. However this requires an extra layer of pointer indirection and a different layout, which is why sorting in Go uses an interface for the container with a Swap method instead of taking a slice of a `Comparable` interface, because it would require allocating an entire new slice of the interface types and then it would only sort that and not the original slice!

Object-oriented programming

Object oriented programming is a language feature that makes good use of the power of vtables. Instead of having separate interface objects that contain the vtables, object-oriented languages like Java just have a vtable pointer at the start of every object. Java-like languages have a system of inheritance and interfaces that can be implemented entirely with these object vtables.

As well as providing additional features, embedding vtables in every object also solves the earlier problem of needing to construct new interface types with indirection. Unlike `Go`, in Java the sorting function can just use the `Comparable` interface on types that implement it.

Reflection

Once you have vtables, it's not too difficult to have the compiler also generate tables of other type information like field names, types and locations. This allows accessing all the data in a type with the same code that can inspect all the data in any other type. This can be used to add a "reflection" feature to your language which can be used to implement things like serialization and pretty printing for arbitrary types. As an extension of the boxing paradigm it has the same tradeoff that it only requires one copy of the code but requires a lot of slow dynamic lookups, which can lead to slow serialization performance.

Examples of languages with reflection features they use for serialization and other things include Java, C# and Go.

Dynamically typed languages

Reflection is very powerful and can do a lot of different metaprogramming tasks, but one thing it can't do is create new types or edit the type information of existing values. If we add the ability to do this, as well as make the default access and modification syntaxes go through reflection, we end up with dynamically typed languages! The incredibly flexibility to do metaprogramming in languages like Python and Ruby comes from effectively super-powered reflection systems that are used for everything.

"But Tristan, that's not how dynamic languages work, they just implement everything with hash tables!" you may say. Well, hash tables are just a good data structure for implementing editable type information tables! Also, that's just how some interpreters like CPython do things. If you look at how a high performance JIT like V8 implements things, it looks a lot like vtables and reflection info! V8's hidden classes (vtables and reflection info) and object layout are similar to what you might see in a Java VM, just with the capability for objects to change to a new vtable at runtime. This is not a coincidence because nothing is ever a coincidence: The person listed on Wikipedia as the creator of V8 previously worked on a high-performance Java VM.

Dictionary Passing

Another way of implementing dynamic interfaces than associating vtables with objects is to pass a table of the required function pointers along to generic functions that need them. This approach is in a way similar to constructing Go-style interface objects at the call site, just that the table is passed as a hidden argument instead of packaged into a bundle as one of the existing arguments.

This approach is used by Haskell type classes although GHC has the ability to do a kind of monomorphization as an optimization through inlining and specialization. Dictionary passing is also used by OCaml with an explicit argument in the form of first class modules, but there's a proposal to add a mechanism to make the parameter implicit.

Swift Witness Tables

Swift makes the interesting realization that by using dictionary passing and also putting the size of types and how to move, copy and free them into the tables, they can provide all the information required to work with any type in a uniform way without boxing them. This way Swift can implement generics without monomorphization and without allocating everything into a uniform representation! They still pay the cost of all the dynamic lookups that all boxing-family implementations pay, but they save on the allocation, memory and cache-incoherency costs. The Swift compiler also has the ability to specialize (monomorphize) and inline generics within a module and across modules with functions annotated `@inlinable` to avoid these costs if it wants to, presumably using heuristics about how much it would bloat the code.

This functionality also explains how Swift can implement ABI stability in a way that allows adding and rearranging fields in `struct`s, although they provide a `@frozen` attribute to opt out of dynamic lookups for performance reasons.

Intensional Type Analysis

One more way to implement interfaces for your boxed types is to add a type ID in a fixed part of the object like where a vtable pointer would go, then generate functions for each interface method that effectively have a big `switch` statement over all the types that implement that interface method and dispatch to the correct type-specific method.

I'm not aware of any languages that use this technique, but C++ compilers and Java VMs do something similar to this when they use profile-guided optimization to learn that a certain generic call site mostly acts on objects of certain types. They'll replace the call site with a check for each common type and then a static dispatch for that common type, with the usual dynamic dispatch as a fallback case. This way the branch predictor can predict the common case branch will be taken and continue dispatching instructions through the static call.

Monomorphization

Now, the alternative approach to boxing is monomorphization. In the monomorphization approach we need to find some way to output multiple versions of our code for each type we want to use it with.

Compilers have multiple phases of representations that the code passes through as it is compiled, and we theoretically can do the copying at any of these stages.

Generating source code

The simplest approach to monomorphization is to do the copying at the stage of the first representation: source code! This way the compiler doesn't even have to have generics support in it, and this is what users of languages like C and Go, where the compiler doesn't support generics, sometimes do.

In C you can use the preprocessor and define your data structure in a macro or a header that you include multiple times with different `#define`s. In Go there are scripts like genny that make this code generation process easy.

The downside of this is that duplicating source code can have a lot of warts and edge cases to look out for depending on the language, and also gives the compiler lots of extra work to do parsing and type checking basically the same code many times. Again depending on language and tools this method's generics can be ugly to write and use, like how if you write one inside a C macro every line has to end with a backslash and all type and function names need to have the type name manually concatenated onto their identifiers to avoid collisions.

D string mixins

Code generation does have something going for it though, which is that you can generate the code using a fully powered programming language, and also it uses a representation that the user already knows.

Some languages that implement generics in some other way also include a clean way of doing code generation to address more general metaprogramming use cases not covered by their generics system, like serialization. The clearest example of this is D's string mixins which enable generating D code as strings using the full power of D during the middle of a compile.

Rust procedural macros

A similar example but with a representation only one step into the compiler is Rust's procedural macros, which take token streams as input and output token streams, while providing utilities to convert token streams to and from strings. The advantage of this approach is that token streams can preserve source code location information. A macro can directly paste code the user wrote from input to output as tokens, then if the user's code causes a compiler error in the macro output the error message the compiler prints will correctly point to the file, line and columns of the broken part of the user's code, but if the macro generates broken code the error message will point to the macro invocation. For example if you use a macro that wraps a function in logging calls and make a mistake in the implementation of the wrapped function, the compiler error will point directly to the mistake in your file, rather than saying the error occurred in code generated by the macro.

Syntax tree macros

Some languages do take the step further and offer facilities for consuming and producing Abstract Syntax Tree (AST) types in macros written in the language. Examples of this include Template Haskell, Nim macros, OCaml PPX and nearly all Lisps.

One problem with AST macros is that you don't want to require users to learn a bunch of functions for constructing AST types as well as the base languages. The Lisp family of languages address this by making the syntax and the AST structure very simple with a very direct correspondence, but constructing the structures can still be tedious. Thus, all the languages I mention have some form of "quote" primitive where you provide a fragment of code in the language and it returns the syntax tree. These quote primitives also have a way to splice syntax tree values in like string interpolation. Here's an example in Template Haskell:

```
-- using AST construction functions
genFn :: Name -> Q Exp
genFn f = do
  x <- newName "x"
  lamE [varP x] (appE (varE f) (varE x))

-- using quotation with $() for splicing
genFn' :: Name -> Q Exp
genFn' f = [| \x -> $(varE f) x |]
```

One disadvantage of doing procedural macros at the syntax tree level instead of token level is that syntax tree types often change with the addition of new language features, while token types can remain compatible. For example OCaml's PPX system needs special infrastructure to migrate parse trees to and from the language version used by a macro. Whereas Rust has libraries that add parsing and quotation utilities so you can write procedural macros in a style similar to syntax tree macros. Rust even has an experimental library that tries to replicate the interface provided by reflection!

Templates

The next type of generics is just pushing the code generation a little further in the compiler. Templates as found in C++ and D are a way of implementing generics where you can specify "template parameters" on types and functions and when you instantiate a template with a specific type, that type is substituted into the function, and then the function is type checked to make sure that the combination is valid.

```
template <class T> T myMax(T a, T b) {
  return (a>b?a:b);
}

template <class T> struct Pair {
  T values[2];
};

int main() {
```

```
  myMax(5, 6);
  Pair<int> p { {5,6} };
  // This would give us a compile error inside myMax
  // about Pair being an invalid operand to `>`:
  // myMax(p, p);
}
```

One problem with the template system is that if you include a template function in your library and a user instantiates it with the wrong type they may get an inscrutable compile error inside your library. This is very similar to what can happen with libraries in dynamically typed languages when a user passes in the wrong type. D has an interesting solution to this which is similar to what popular libraries in dynamic languages do: just use helper functions to check the types are valid, the error messages will clearly point to the helpers if they fail! Here's the same example in D, note the `if` in the signature and the generally better syntax (`!` is how you provide template parameters):

```
// We're going to use the isNumeric function in std.traits
import std.traits;

// The `if` is optional (without it you'll get an error inside like C++)
// The `if` is also included in docs and participates in overloading!
T myMax(T)(T a, T b) if(isNumeric!T) {
    return (a>b?a:b);
}

struct Pair(T) {
  T[2] values;
}

void main() {
  myMax(5, 6);
  Pair!int p = {[5,6]};
  // This would give a compile error saying that `(Pair!int, Pair!int)`
  // doesn't match the available instance `myMax(T a, T b) if(isNumeric!T)`:
  // myMax(p, p);
}
```

C++20 has a feature called "concepts" that serves the same purpose except with a design more like defining interfaces and type constraints.

Compile time functions

D's templates have a number of extensions that allow you to use features like compile time function evaluation and `static if` to basically make templates act like functions that take a compile time set of parameters and return a non-generic runtime function. This makes D templates into a fully featured metaprogramming system, and as far as I understand modern C++ templates have similar power but with less clean mechanisms.

There's some languages that take the "generics are just compile time functions" concept and run with it even further, like Zig:

```
fn Stack(comptime T: type) type {
    return struct {
        items: []T,
        len: usize,

        const Self = @This();
        pub fn push(self: Self, item: T) {
            // ...
        }
    };
}
```

Zig does this using the same language at both compile time and runtime, with functions split up based on parameters marked `comptime`. There's another language that uses a separate but similar language at the meta level called Terra. Terra is a dialect of Lua that allows you to construct lower level C-like functions inline and then manipulate them at the meta level using Lua APIs as well as quoting and splicing primitives:

```
function MakeStack(T)
    local struct Stack {
        items : &T; -- &T is a pointer to T
        len : int;
    }
    terra Stack:push(item : T)
        -- ...
    end
    return Stack
end
```

Terra's crazy level of metaprogramming power allows it to do things like implement optimizing compilers for domain specific languages as simple functions, or implement the interface and object systems of Java and Go in a library with a small amount of code. Then it can save out generated runtime-level code as dependency-free object files.

Rust generics

The next type of monomorphized generics of course moves the code generation one step further into the compiler, after type checking. I mentioned that the type of inside-the-library errors you can get with C++ are like the errors you can get in a dynamically typed language, this is of course because there's basically only one type of type in template parameters, like a dynamic language. So that means we can fix the problem by adding a type system to our meta level and having multiple types of types with static checking that they support the operations you use. This is how generics work in Rust, and at the language level also how they work in Swift and Haskell.

In Rust you need to declare "trait bounds" on your type parameters, where `trait`s are like interfaces in other languages and declare a set of functionality provided by the type. The Rust compiler will check that the body of your generic functions will work with any type conforming to your trait bounds, and also not allow you to use functionality of the type not declared by the trait bounds. This way users of generic functions in Rust can *never* get compile errors inside a library function when they instantiate it. The compiler also only has to type check each generic function once.

```
fn my_max<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}

struct Pair<T> {
    values: [T; 2],
}

fn main() {
    my_max(5,6);
    let p: Pair<i32> = Pair { values: [5,6] };
    // Would give a compile error saying that
    // PartialOrd is not implemented for Pair<i32>:
    // my_max(p,p);
}
```

At the language level this is very similar to the kind of type system you need to implement generics with interface support using the boxing approach to generics, which is why Rust can support both using the same system! Rust 2018 even added a uniform syntax where a `v: &impl SomeTrait` parameter gets monomorphized but a `v: &dyn SomeTrait` parameter uses boxing. This property also allows compilers like Swift's and Haskell's GHC to monomorphize as an optimization even though they default to boxing.

Machine code monomorphization

The logical next step in monomorphized generics models is pushing it further in the compiler, after the backend. Just like we can copy source code templates that are annotated with placeholders for the generic type, we can generate machine code with placeholders for the type-specific parts. Then we can stamp these templates out very quickly with a `memcpy` and a few patches like how a linker works! The downside is that each monomorphized copy couldn't be specially optimized by the optimizer, but because of the lack of duplicate optimization, compilation can be way faster. We could even make the code stamper a tiny JIT that gets included in binaries and stamps out the monomorphized copies at runtime to avoid bloating the binaries.

Actually I'm not aware of any language that works this way, it's just an idea that came to me while writing as a natural extension of this taxonomy, which is exactly the kind of thing I hoped for from this exercise! I hope this post gives you a clearer picture of the generics systems in different languages and how they can be fit together into a coherent taxonomy, and prompts you to think about the directions in concept-space where we might find new cool programming languages.

Tristan Hume
Programming enthusiast

github.com/trishume
tristan@thume.ca
@trishume

